

Вопросы к курсу «Базы данных 5 семестр»

1. Определение представления. Назначение.
2. Операции над представлениями. Пример.
3. Определение триггера. Назначение, Способ функционирования.
4. Операции над триггерами.
5. Аномалия потери изменений.
6. Аномалия грязного чтения.
7. Аномалия неповторяющегося чтения.
8. Аномалия чтения фантомов.
9. Определение транзакции. Требования ACID к транзакциям.
10. Синтаксис операторов управления транзакциями.
11. Выполнение транзакции: фиксация изменений или откат.
12. Блокировки на чтение и на запись. Взаимодействие блокировок.
13. Уровни изоляции и исключение проблем совместного доступа.
14. Явные и неявные блокировки.
15. Место БД в системе управления. Схема. Цель управления.
16. Уровни абстракции в архитектуре СУБД. Уровень подсхем, концептуальный и физический уровни.
17. Цикл выполнения запроса. Алгоритм выполнения запроса на выборку.
18. Системный каталог. Назначение. Хранимые данные.
19. Пример системного каталога Information_schema.
20. Сетевые архитектуры. Классификация. Типы архитектур. Составные части информационной системы.
21. Современная сетевая архитектура Клиент сервер. Схема.
22. Понятие сбоя в информационной системе. Типы сбоев. Мягкие и жесткие сбои.
23. Резервное копирование Дамп базы данных. Назначение. Место хранения. Периодичность выполнения.
24. Журнал. Назначение журнала. Метод сохранения данных в журнале. Восстановление после мягкого сбоя.
25. Разграничение доступа. Операторы создания пользователей, назначения и отмены прав доступа.
26. Функции и процедуры Назначение. Синтаксис вызова.
27. Операторы присваивания, программных скобок и объявления переменных.
28. Операторы ветвления.
29. Операторы циклов.
30. Индексы. Назначение. Синтаксис команд управления индексами.

1. Определение представления. Назначение.

Представление (View) — это виртуальная таблица, которая формируется на основе результата запроса. Представления не хранят сами данные, они хранят только запрос, который определяет, какие данные должны быть извлечены из одной или нескольких таблиц.

Особенности:

- является виртуальной таблицей;
- может быть использовано в других запросах;
- доступно, в основном, для чтения.

Назначение:

- хранение сложных запросов;
- представление данных в виде, удобном пользователю;
- сокрытие конфиденциальной информации;
- разграничение доступа к данным.

Ограничения представлений. Нельзя:

- повесить триггер;
- использовать временные таблицы в качестве источника;
- сделать временное представление;
- включить подзапрос во From.

Представления могут основываться как на таблицах, так и на других представлениях.

Представление называется обновляемым, если к нему могут быть применимы операторы UPDATE и DELETE для изменения данных в таблицах, на которых основано представление. Такие представления имеют ряд ограничений: они не должны содержать JOIN, GROUP BY, HAVING, WHERE и тд, поэтому они редко используются.

Требования к обновляемости:

- связь 1 к 1 с исходными таблицами (т.е. каждой строке представления должно соответствовать по одной строке в таблицах-источника);
- отсутствие вычисляемых полей.

Обновляемое представление может допускать добавление данных (INSERT), если все поля таблицы-источника, не присутствующие в представлении, имеют значения по умолчанию. [\[1\]](#)

Алгоритмы представлений:

- MERGE — при обращении к представлению используются ссылки на исходные таблицы
- TEMPTABLE — создается новая временная таблица, обращение к ней

2. Операции над представлениями. Пример.

Операции:

- **CREATE VIEW** - создание представления

Пример:

```
CREATE VIEW employee_details AS
```

```
SELECT e.employee_id, e.name, d.department_name, e.salary FROM employees e  
JOIN departments d ON e.department_id = d.department_id;
```

- **ALTER VIEW** - изменение представления

Пример:

```
ALTER VIEW employee_details AS
```

```
SELECT e.employee_id, e.name, d.department_name, e.salary FROM employees e  
JOIN departments d ON e.department_id = d.department_id  
WHERE d.department_id = 'Sales';
```

- **DROP VIEW**

Пример:

```
DROP VIEW employee_details;
```

1. Создание представления:

```
CREATE  
[OR REPLACE]  
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]  
[DEFINER = user]  
[SQL SECURITY { DEFINER | INVOKER }]  
VIEW view_name [(column_list)]  
AS select_statement  
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

```
CREATE [OR REPLACE] VIEW <name>  
[( <field> )] AS <Select> [WITH CHECK OPTION];
```

2. Изменение представления:

```
ALTER  
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]  
[DEFINER = user]  
[SQL SECURITY { DEFINER | INVOKER }]  
VIEW view_name [(column_list)]  
AS select_statement  
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

```
ALTER VIEW <name> [( <field> )] AS  
<Select> [WITH CHECK OPTION];
```

3. Удаление представления:

```
DROP VIEW [IF EXISTS]  
view_name [, view_name] ...  
[RESTRICT | CASCADE]
```

```
DROP VIEW <name>
```

хз добавлять ли примеры из презы...

3. Определение триггера. Назначение, Способ функционирования.

Триггер — процедура, которая автоматически запускается при возникновении определенных событий.

Событие триггера — событие, управляющее запуском триггера, описывается в виде логических условий.

Особенности:

- выполняется автоматически при наступлении события;
- содержит только команды ЯМД (язык модификации данных, т. е. insert, update, delete).

Момент выполнения:

- до выполнения события Before;
- после выполнения события After.

Назначение:

- проверка сложных ограничений целостности;
- автоматизация обработки данных;
- аудит действий пользователей;
- установка начальных значений при добавлении данных в таблицы;
- проверка прав доступа.

INSERT | DELETE | UPDATE — событие триггера.

Событием триггера может быть одна команда или любая комбинация указанных команд.

BEFORE | AFTER — время срабатывания триггера: перед выполнением события триггера или после него.

Ограничения целостности проверяются во время выполнения события триггера.

ON <имя таблицы> — таблица, к которой привязан триггер.

FOR EACH ROW — область действия триггера для каждой строки.

4. Операции над триггерами.

Операции:

- **CREATE TRIGGER** - создание триггера

Пример:

```
CREATE TRIGGER after_employee_insert
```

```
AFTER INSERT ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO audit_log (action, employee_id, name, position, salary)
```

```
    VALUES ('INSERT', NEW.employee_id, NEW.name, NEW.position, NEW.salary);
```

```
END;
```

- **DROP TRIGGER** - удаление триггера

Пример:

```
DROP TRIGGER trigger_name;
```

- Просмотр существующих триггеров

SHOW TRIGGERS;

5. Аномалия потери изменений.

Аномалия — это неправильное, нежелательное или несогласованное состояние данных, которое возникает из-за неконтролируемого или недостаточно изолированного взаимодействия нескольких параллельных транзакций.



OO — единственная копия кортежа

T1 — изменяет объект OO на O1

T2 — изменяет объект OO на O2

T2 — откатывает объект O2 на OO

T1 — читает объект O, ждет O1, получает OO

Аномалия: транзакция T1 не видит свои изменения объекта O.

СУБД не допускает взаимовлияния транзакций с потерей изменений.

Пример:

транзакция 1 — `UPDATE employees SET salary = 39200 WHERE employees_id = 123;`

транзакция 2 — `UPDATE employees SET position = "new pos" WHERE employees_id = 123;`

транзакция 2 — `ROLLBACK;`

транзакция 1 — `SELECT * FROM employees WHERE employees_id = 123;`

итог: в записи ничего не поменялось: ни поле salary, ни поле position.

Проблема — нет изолированности пользователей.

Для ее решения достаточно заблокировать доступ на чтение после изменения объекта O.

6. Аномалия грязного чтения.

Аномалия — это неправильное, нежелательное или несогласованное состояние данных, которое возникает из-за неконтролируемого или недостаточно изолированного взаимодействия нескольких параллельных транзакций.



О0 — единственная копия кортежа

T1 — изменяет объект О0 на О1

T2 — читает объект О и видит О1

T1 — откатывает объект О1 на О0

T2 — продолжает считать, что О равно О1, а оно О0

Аномалия: транзакция T2 читает и работает с «грязными» данными

Пример:

транзакция 1 — `UPDATE employees SET salary = 39200 WHERE employees_id = 123;`

транзакция 2 — `SELECT * FROM employees WHERE employees_id = 123;`

транзакция 1 — `ROLLBACK;`

итог: транзакция 2 содержит неактуальные данные.

Проблема — нет изолированности пользователей.

Для ее решения достаточно заблокировать доступ на чтение после изменения объекта О.

7. Аномалия неповторяющегося чтения.

Аномалия — это неправильное, нежелательное или несогласованное состояние данных, которое возникает из-за неконтролируемого или недостаточно изолированного взаимодействия нескольких параллельных транзакций.



О0 — единственная копия кортежа

T1 — читает объект О и получает О0

T2 — меняет объект О и пишет О1

T2 — сохраняет изменения транзакции — в базе О1

T1 — читает О и получает О1, а оно было О0

Аномалия: транзакция T1 при повторном чтении получает иной результат.

Пример:

транзакция 1 — *SELECT * FROM employees WHERE employees_id = 123;*

транзакция 2 — *UPDATE employees SET salary = 39200 WHERE employees_id = 123;*

транзакция 2 — COMMIT;

транзакция 1 — *SELECT * FROM employees WHERE employees_id = 123;*

итог: транзакция 1 получает разные данные при одинаковом запросе.

Проблема — нет изолированности пользователей.

Для ее решения достаточно заблокировать доступ на чтение после изменения объекта О.

8. Аномалия чтения фантомов.

Аномалия — это неправильное, нежелательное или несогласованное состояние данных, которое возникает из-за неконтролируемого или недостаточно изолированного взаимодействия нескольких параллельных транзакций.



Tab — единственная копия таблицы

T1 — читает Tab и получает n записей

T2 — добавляет в Tab1 новую запись

T2 — сохраняет изменения транзакции — в базе

T1 — читает Tab и получает n+1 записей, было n записей

Аномалия: транзакция T1 при повторном чтении получает иное число записей.

Пример:

транзакция 1 — *SELECT * FROM employees;*

транзакция 2 — *INSERT INTO employees SET VALUES (124, "some name", "some position", 27000);*

транзакция 2 - COMMIT;

транзакция 1 — *SELECT * FROM employees;*

итог: транзакция 1 получает разное количество записей в таблице при одинаковом запросе.

Проблема — нет изолированности операций.

Для ее решения требуется блокировка таблицы целиком.

9. Определение транзакции. Требования ACID к транзакциям.

Транзакция — это упорядоченная последовательность операторов обработки данных, которая переводит базу данных из одного согласованного состояния в другое.

Требования к транзакциям:

- Atomicity (атомарность) — выполняются либо все операции, входящие в транзакцию, либо ни одной;
- Consistency (согласованность) — транзакция начинается на согласованном множестве данных и после её завершения множество данных согласовано;
- Isolation (изолированность) — отсутствие влияния транзакций друг на друга;
- Durability (долговечность) — результаты завершённой транзакции не могут быть потеряны.

Транзакции, удовлетворяющие этим свойствам, называют **ACID-транзакциями**.

10. Синтаксис операторов управления транзакциями.

- *START TRANSACTION*: начинает новую транзакцию.
- *COMMIT*: фиксирует все изменения, сделанные в рамках текущей транзакции.
- *ROLLBACK*: откатывает все изменения, сделанные в рамках текущей транзакции.
- *SAVEPOINT savepoint_name*: создаёт точку сохранения внутри транзакции, к которой можно откатиться.
- *RELEASE SAVEPOINT savepoint_name*: удаляет точку сохранения, созданную командой *SAVEPOINT*.
- *ROLLBACK TO SAVEPOINT savepoint_name*: откатывает транзакцию до указанной точки сохранения.

START TRANSACTION;

... (модификация данных)

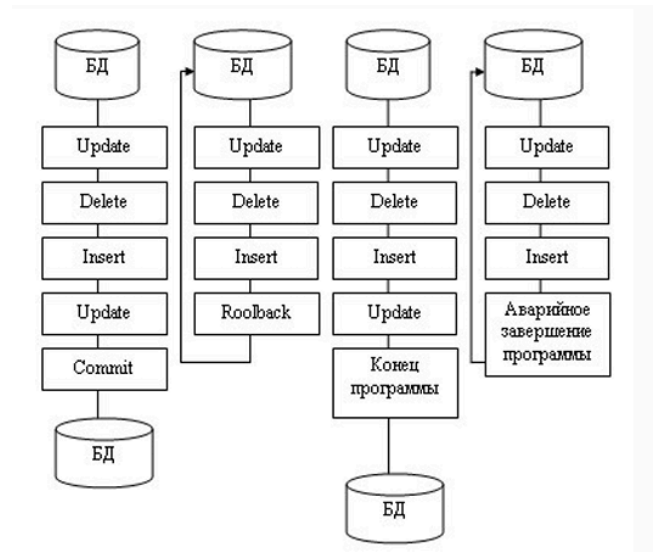
COMMIT;

11. Выполнение транзакции: фиксация изменений или откат.

Начало транзакции — при появлении первого оператора.

Завершение транзакции:

- поступила команда commit — фиксация.
- поступила команда rollback — откат.
- успешно завершена программа (exit, quit) — фиксация.
- программа завершена аварийно (abort) — откат.



Журнал транзакций содержит:

- номер транзакции;
- состояние транзакции: завершена фиксацией или откатом, не завершена, находится в состоянии ожидания;
- точки сохранения (явные и неявные);
- команды, составляющие транзакцию.

Фиксация транзакции:

1. Изменения, внесенные транзакцией, помечаются как постоянные.

2. Удаляются точки сохранения транзакции.
3. Освобождаются объекты, заблокированные транзакцией.
4. В журнале транзакций транзакция помечается как завершенная.

Откат транзакции:

** выполняется медленнее фиксации*

1. Считываются старые значения и возвращаются в БД.

Идеальное выполнение транзакций — они полностью изолированы друг от друга.

Транзакции не влияют друг на друга, если:

1. они только читают данные;
2. они обращаются к разным данным.

Взаимовлияние транзакций может проявляться в виде:

1. потери изменений;
2. чернового (грязного) чтения;
3. неповторяемого чтения;
4. фантомов.

12. Блокировки на чтение и на запись. Взаимодействие блокировок.

Блокировка — это временное ограничение доступа к данным, участвующим в транзакции, со стороны других транзакций. Т.е. блокировка накладывается на данные транзакцией, которая их использует.

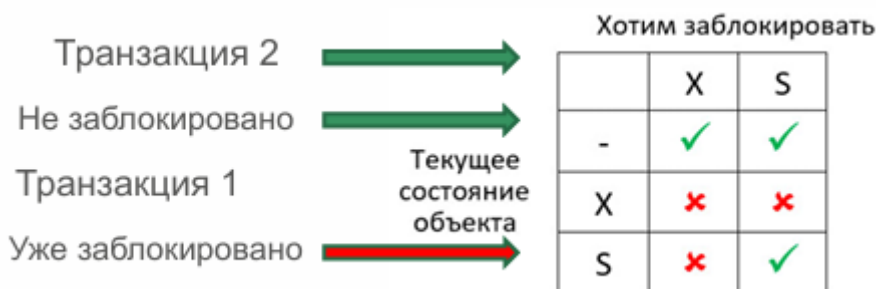
СУБД обеспечивает блокировки двух типов:

- **S-блокировка** (shared, блокировка по чтению) — разделяющая блокировка или блокировка, допускающая чтение. Обеспечивает совместный доступ на чтение объекта.
- **X-блокировка** (exclusive, блокировка по записи) — эксклюзивная блокировка или блокировка для записи. Не допускает совместный доступ к объекту даже на чтение.

S-блокировки одного и того же объекта несколькими транзакциями совместимы — нескольким транзакциям допускается одновременно читать один и тот же объект.

X-блокировки одного и того же объекта разными транзакциями не совместимы — никакой транзакции нельзя изменять объект, изменяемый некоторой транзакцией.

Если одна транзакция читает данные и удерживает на них блокировку на чтение, то другая транзакция не может одновременно записывать (изменять) эти же данные — и наоборот. Так происходит, потому что запись меняет данные, а чтение должно видеть согласованное состояние.



Режимы блокировок:

- неявные блокировки при выполнении транзакций;
- явные блокировки — сделанные из кода.

13. Уровни изоляции и исключение проблем совместного доступа.

В СУБД используются методы регулирования совместного выполнения транзакций.

Пусть T₁ и T₂ — одновременно выполняемые транзакции. Требуется построить сериальный план, при котором **последовательно-параллельное** транзакций будет эквивалентно **последовательному**.

Возможные конфликты между T₁ и T₂:

Вид конфликта	Описание	Проблема
W/W (конфликт Write/Write)	транзакция T2 пытается изменить объект, изменённый не закончившейся транзакцией T1	потерянных изменений
R/W (конфликт Read/Write)	транзакция T2 пытается изменить объект, прочитанный не закончившейся транзакцией T1	неповторяющиеся чтения
W/R (конфликт Write/Read)	транзакция T2 пытается читать объект, изменённый не закончившейся транзакцией T1	«грязного» чтения

Читающие транзакции не конфликтуют!

Уровни изоляции в ISO/ANSI:

Уровень изоляции	Грязное чтение	Неповторяемое чтение	Фантомы
Read Uncommitted – чтение незавершённых транзакций	да	да	да
Read Committed – чтение завершённых транзакций	нет	да	да
Repeatable Read – повторяемое чтение	нет	нет	да
Serializable – последовательное чтение	нет	нет	нет

Но не во всех СУБД реализовано именно так.

READ UNCOMMITTED:

Измененные данные доступны для чтения до подтверждения изменений, то есть до вызова COMMIT.

READ COMMITED:

Допускает чтение только зафиксированных данных, то есть только тех изменений, которые подтверждены COMMIT. Важно, что внутри транзакции, которая совершает

изменения в таблице, изменения тоже не будут отображаться до вызова COMMIT. То есть вызов SELECT до вызова COMMIT не покажет внесенные изменения.

REPEATABLE READ:

Уровень изоляции, используемый MySQL по умолчанию. Гарантирует, что данные, прочитанные в начале транзакции, не изменятся.

Отличается от READ COMMITED тем, что добавленные/измененные данные будут видны и доступны уже внутри транзакции, но не будут доступны извне до вызова COMMIT.

SERIALIZABLE:

Транзакции выполняются так, будто они выполняются последовательно. MySQL блокирует каждую строку, над которой происходит какое-либо действие.

Синтаксис выбора уровня изоляции:

SET [SESSION | GLOBAL] **TRANSACTION ISOLATION LEVEL** уровень_изоляции;

14. Явные и неявные блокировки.

(повторение из билета 12)

Блокировка — это временное ограничение доступа к данным, участвующим в транзакции, со стороны других транзакций.

СУБД обеспечивает блокировки двух типов:

- S-блокировка (Shared): разделяемая блокировка или блокировка, допускающая чтение. Обеспечивает совместный доступ на чтение объекта.
- X-блокировка (Xclusive): эксклюзивная блокировка или блокировка для записи, не допускающая совместный доступ к объекту даже на чтение. Это безопасный способ гарантировать наличие единственного клиента, который изменяет запись.

Режимы блокировки

1. *Неявные* блокировки при выполнении транзакций.
2. *Явные* блокировки, выполненные с помощью кода.

Цель явных блокировок:

- загрузка данных из файла в таблицу;
- проведение операций, требующих явной блокировки на заданном уровне изоляции.

Синтаксис указания явной блокировки для таблицы (для MySQL):

LOCK TABLES

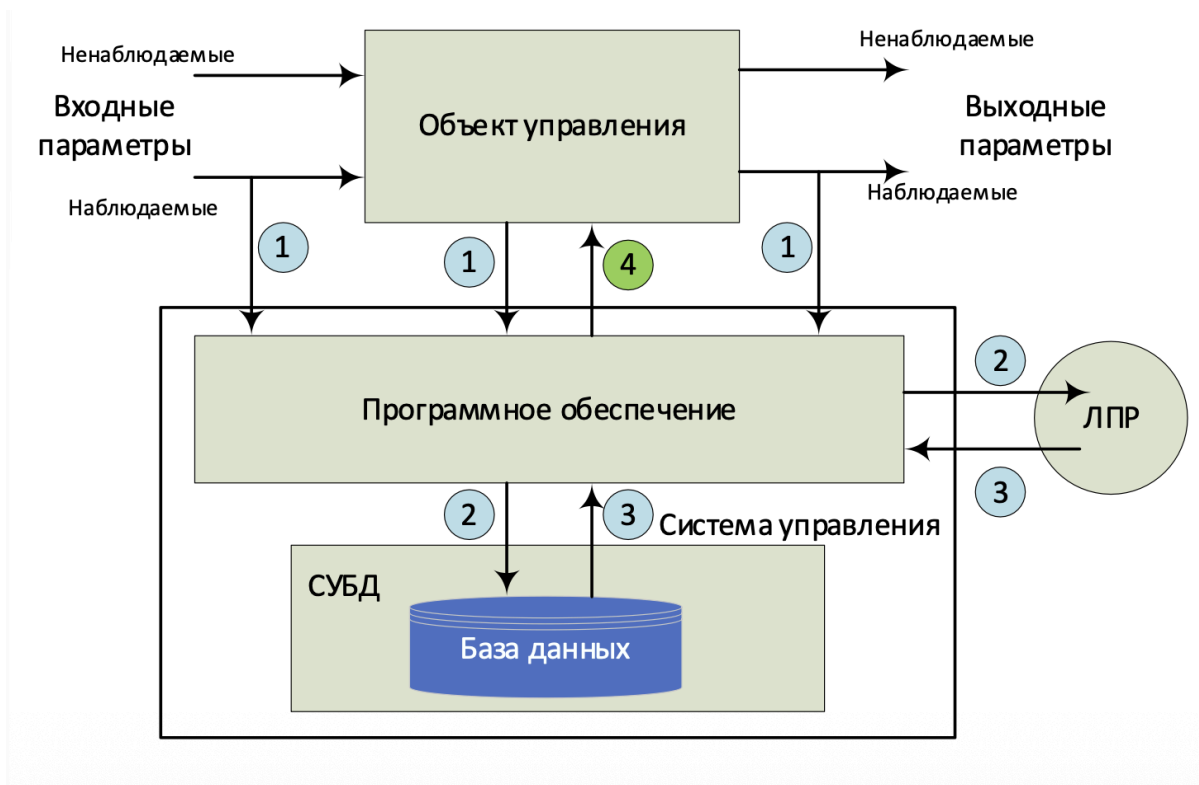
table_name READ,	% блокировка на Чтение
table_name2 WRITE;	% блокировка на Запись

UNLOCK TABLES; % разбан таблицы

MySQL содержит две очереди табличных блокировок: *read lock queue* (очередь блокировок на чтение) и *write lock queue* (очередь блокировок на запись). Очередь блокировок на запись имеет приоритет перед очередью блокировок на чтение, то есть, если очередь блокировок на запись не пуста, то сервер MySQL будет освобождать её, пока она не опустеет, и только потом приступит к очереди блокировок на чтение.

15. Место БД в системе управления. Схема. Цель управления.

Цель управления – обеспечение устойчивого управления для достижения результата



Типа БД является памятью в системе управления, где изначальные входные данные проходят через объект управления в ПО, внутри ПО у нас формируется SQL запрос, который передается СУБД, после чего СУБД возвращает полученные данные в ПО, которое уже приводит их в читаемый вид и отдает объекту управления, который возвращает пользователю данные по его запросу.

ЧТО НАХУЙ ТАКОЕ ЛПР?

ЛПР – лицо принимающее решение

Число функций >0 автоматизированное

Число функций =0 автоматическое

по простому: лпр — человек, который определяет, по какому сценарию нужно пойти, чтобы удержать систему в устойчивом состоянии (между верхней и нижней граней графика). тип в программу поступают данные, но она не знает, что ей с ними делать (она может распорядиться этими данными несколькими разными способами), а человек в зависимости от конечной цели, а также границ устойчивости системы, говорит что именно надо сделать.

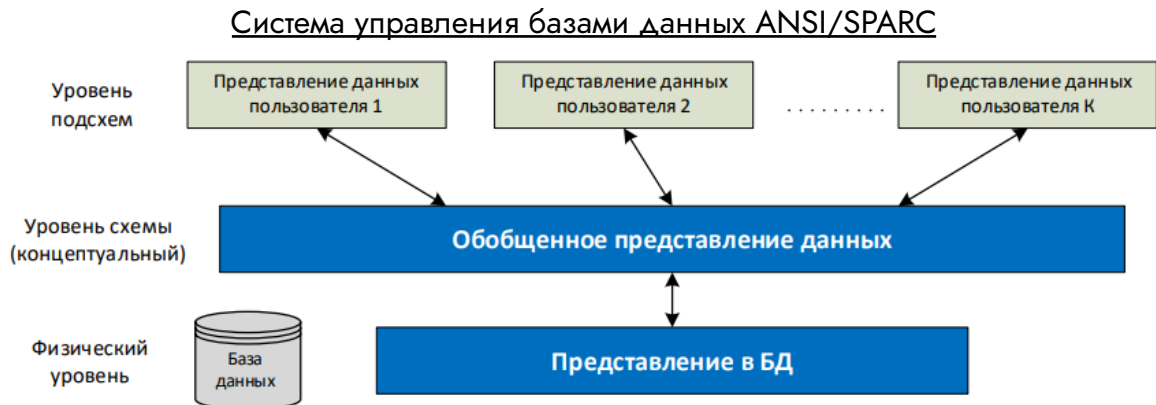
что такое автоматизированное и автоматическое. типом когда у нас поступает 0 функций, то у системы есть только 1 способ того, что делать дальше, а при числе функций >0 у системы появляются разные пути, по которым она может работать с данными (и не

только), как раз поэтому и нужен человек, чтобы ВЫБРАТЬ 1 из способов (путей/сценариев) работы. НУРЕ!

это ты по? да я

мы как опять к тау вернулись

16. Уровни абстракции в архитектуре СУБД. Уровень подсхем, концептуальный и физический уровни.



Подсхема — это представление базы данных конечным пользователем. Уровень подсхем определяется правами доступа.

Поставщики			Детали	
КодП	ИмяП	Город	КодД	Цена
S1	Иванов	Москва	P1	10.00
S2	Петров	Челябинск	P2	40.00
...			...	

Диспетчер

Поставки			
ИмяП	ИмяД	К-во	Цена
Иванов	Болт	3000	10.00
Петров	Гайка	5000	40.00
...			

Экономист

Схема (концептуальный уровень) — обобщенное представление данных. Описывает логическую структуру базы данных и формируется системным администратором.

Поставщики				Детали			
КодП	ИмяП	Город	...	КодД	ИмяД	Цена	...
S1	Иванов	Москва		P1	Болт	10.00	
S2	Петров	Челябинск		P2	Гайка	40.00	
...				...			

Поставки		
КодП	ИмяД	К-во
S1	P1	3000
S2	P2	5000
...		

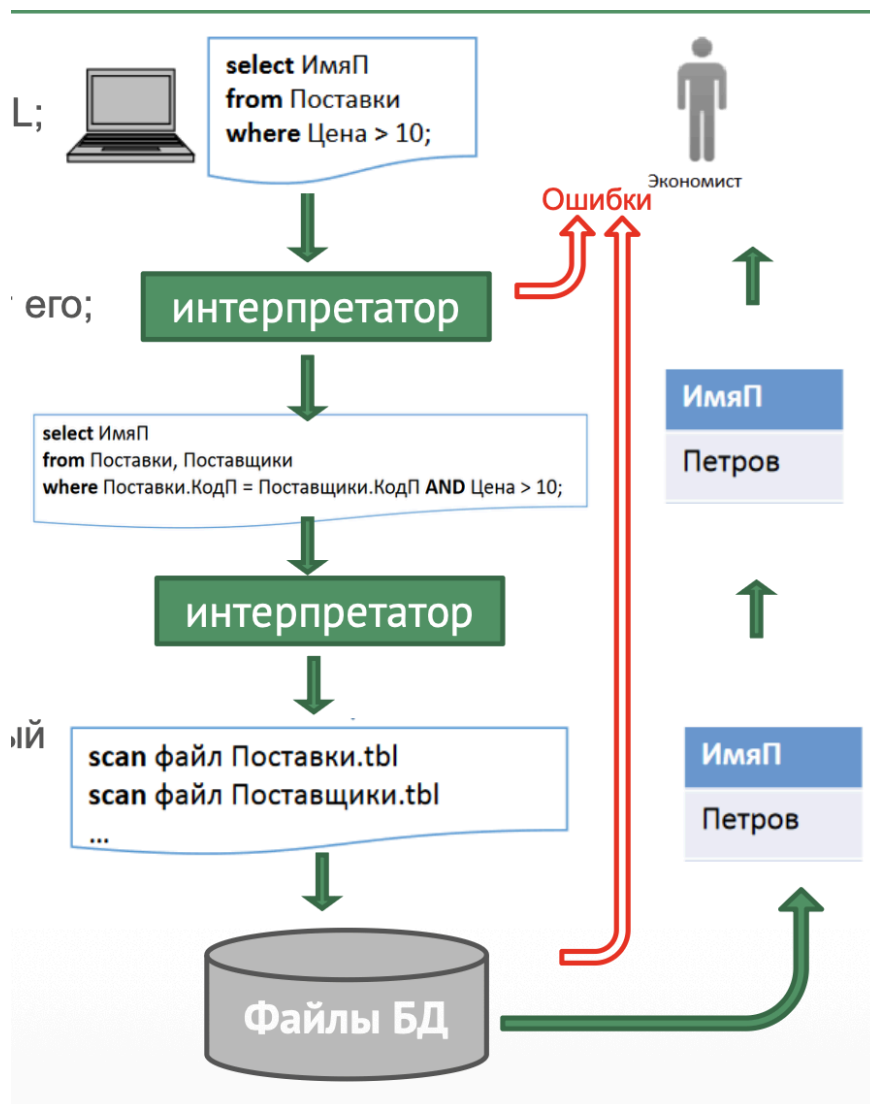
Физический уровень — это представление схемы в виде файлов файловой системы. Этот уровень реализуется средствами конкретной СУБД.

17. Цикл выполнения запроса. Алгоритм выполнения запроса на выборку.

Цикл выполнения запроса (Query Processing Cycle) — это последовательность этапов, которые проходит запрос к базе данных с момента его поступления от пользователя до выдачи готового результата.

Цикл выполнения запроса состоит из следующих шагов:

- 1) пользователь формирует запрос на языке SQL;
- 2) СУБД воспринимает запрос и интерпретирует его;
- 3) запрос к подсхеме преобразуется в эквивалентный запрос к схеме;
- 4) запрос к схеме преобразуется в эквивалентный запрос к физическому представлению;
- 5) обрабатывается запрос к физическому представлению
- 6) результат преобразуется к виду уровня схемы;
- 7) результат преобразуется к виду уровня подсхемы;
- 8) результат помещается в рабочую область пользователя;



Алгоритм выполнения запроса на выборку

18. Системный каталог. Назначение. Хранимые данные.

Системный каталог — это хранилище метаданных базы данных.

В системном каталоге хранятся:

- имена, типы и размеры элементов данных;
- имена связей;
- накладываемые на данные ограничения целостности;
- имена пользователей, которым предоставлено право доступа к данным;
- внешняя, концептуальная и внутренняя схемы и отображения между ними;
- статистические данные (частота транзакций, счетчики обращения к объектам БД).

Системный каталог реализован в системной части СУБД. Доступ осуществляется через **INFORMATION_SCHEMA** — отображение системного каталога для пользователя.

Преимущества наличия ССА

1. Информация о данных может быть централизованно собрана и сохранена.
2. Общий доступ => пользователи могут понять предназначение данных.
3. Интегрированы права на данные.
4. Централизованное хранение => исключается избыточность и противоречивость данных.
5. Внесенные в БД изменения могут быть заprotoколированы.
6. Последствия любых изменений могут быть определены до их внесения.
7. Меры обеспечения безопасности могут быть дополнительно усилены.
8. Появляются новые возможности для организации поддержки целостности данных.
9. Может выполняться аудит данных — процесс изучения и оценки данных на точность, полноту и соответствие требованиям.

19. Пример системного каталога Information_schema.

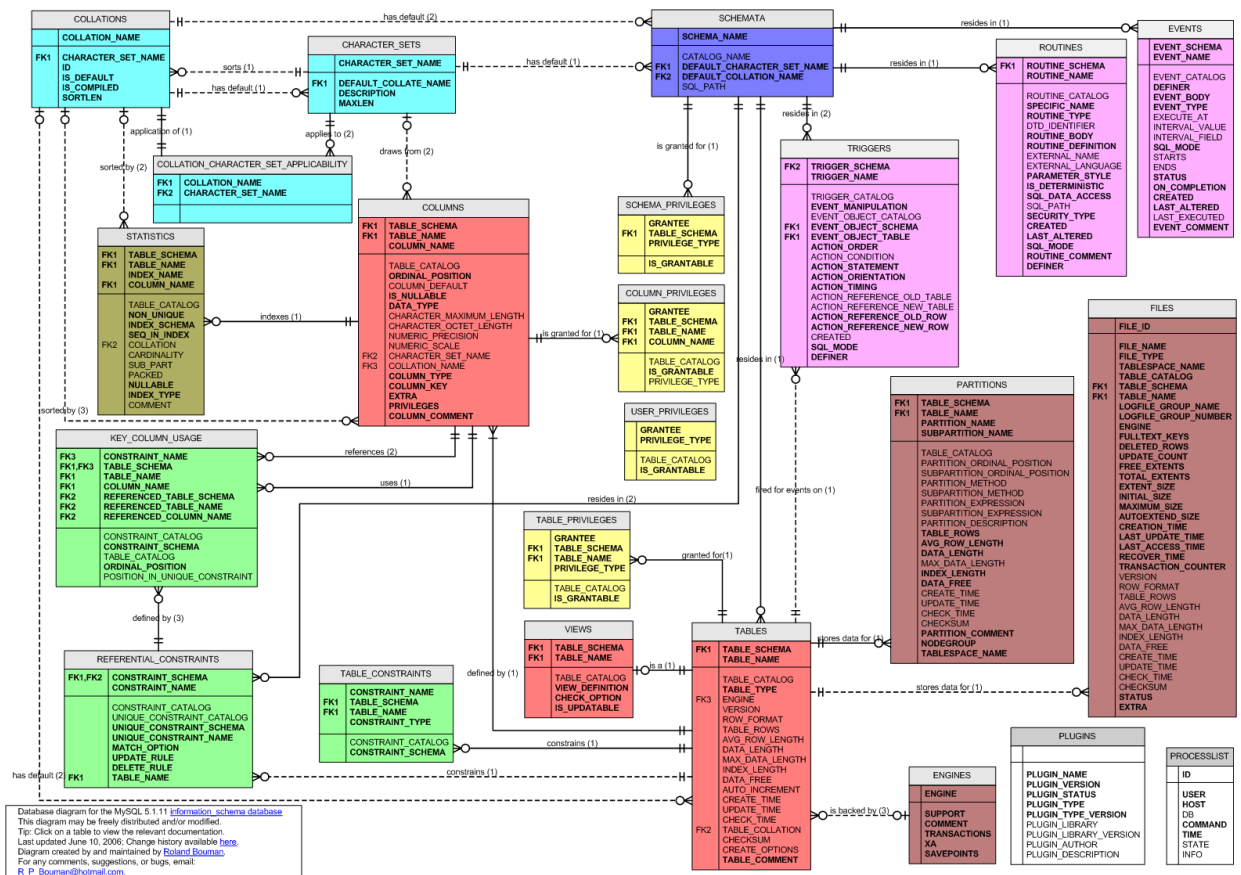
Системный каталог — хранилище метаданных базы данных.

Для пользователя отображением системного каталога является

INFORMATION_SCHEMA. Она построена на естественных составных ключах.

Таблицы в ней различаются по цветам, каждый цвет соответствует хранимым данным в таблице:

- темно-синий — схемы данных;
- светло-синий — наборы символов и последовательности сортировок;
- красный — основные таблицы;
- темно-зеленый — статистика для построения индексов;
- желтый — права пользователей на объекты;
- зеленый — программные компоненты;
- коричневый — физические характеристики таблиц и процедур.



INFORMATION_SCHEMA

Пример запроса к INFORMATION_SCHEMA:

```
select table_name from information_schema.tables where table_schema = 'database_name';
```

Результат данного запроса — будут выведен список таблиц для database_name.

20. Сетевые архитектуры. Классификация. Типы архитектур.

Составные части информационной системы.

Сетевая архитектура — это способ распределения компонентов СУБД между вычислительными узлами сети.

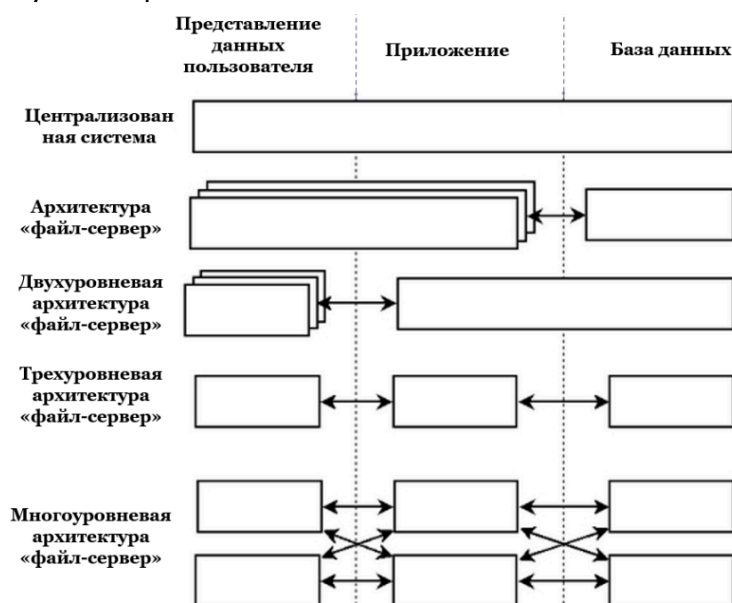
Компоненты системы БД:

- пользовательский интерфейс,
- прикладные функции,
- машина БД,
- система управления файлами.

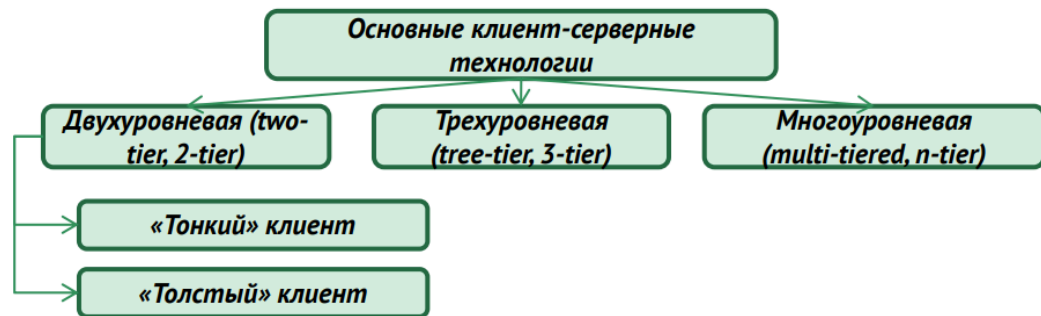
(выжимка из дипсика)

Классификация по степени распределения компонентов

1. **Архитектура с централизованной обработкой данных:** один компьютер (мейнфрейм) выполняет обработку, хранение и управление данными. Доступ для пользователей осуществляется через терминалы доступа, которые только отображают информацию и передают нажатие клавиш.
2. **Архитектура "файл-сервер":** данные хранятся в файлах на выделенном сервере. Сетевая ОС предоставляет доступ к файлам. Приложение (СУБД) целиком выполняется на клиентском ПК. При запросе данных весь файл (например, таблица) копируется по сети на клиент, где происходит его обработка.
3. **Архитектура "клиент-сервер":** имеет двухуровневую, трехуровневую и многоуровневую разновидности. Сервер отвечает за хранение данных, их целостность и выполнение операций обработки данных. Клиент отвечает за бизнес-логику и интерфейс.



Архитектура клиент-сервер

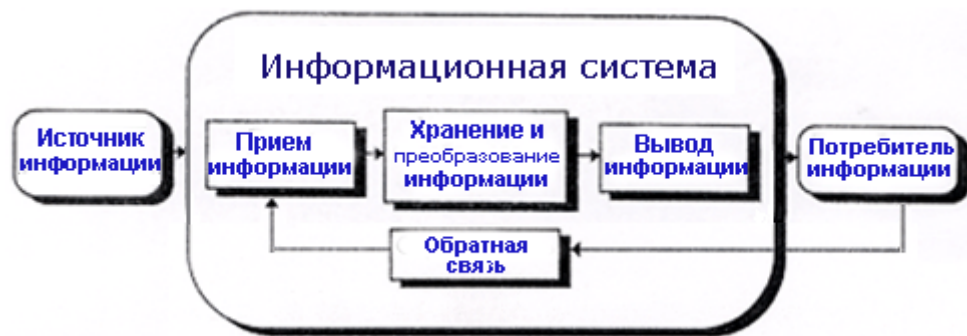


“Толстый” клиент — десктопное приложение, где бизнес-логика и интерфейс устанавливаются и выполняются на компьютере пользователя.

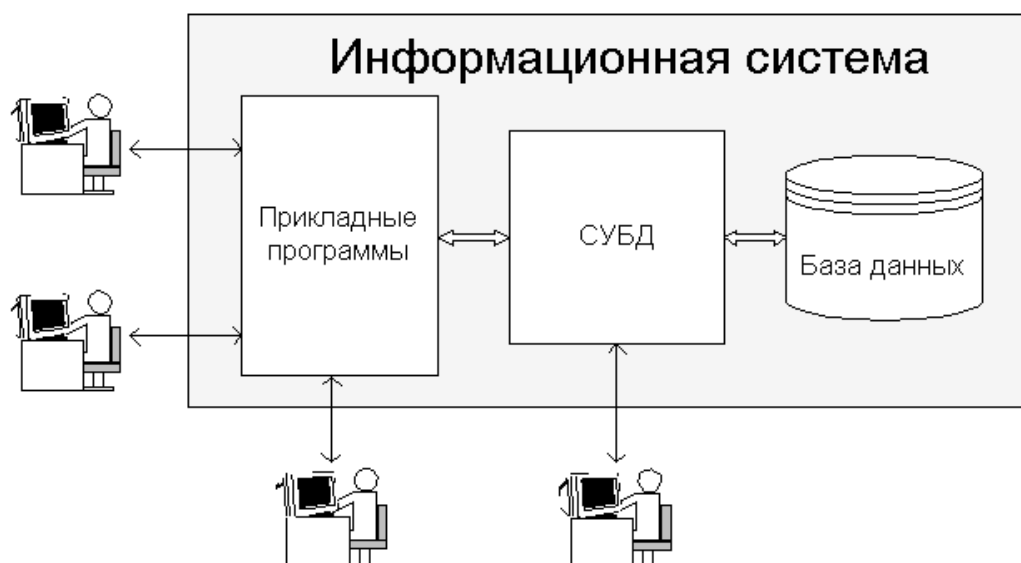
“Тонкий” клиент — веб-браузер или оболочка, вся бизнес-логика выполняется на сервере, клиент только отображает результат.

Составные части информационной системы

вот такая смешная картинка



и вот такая)

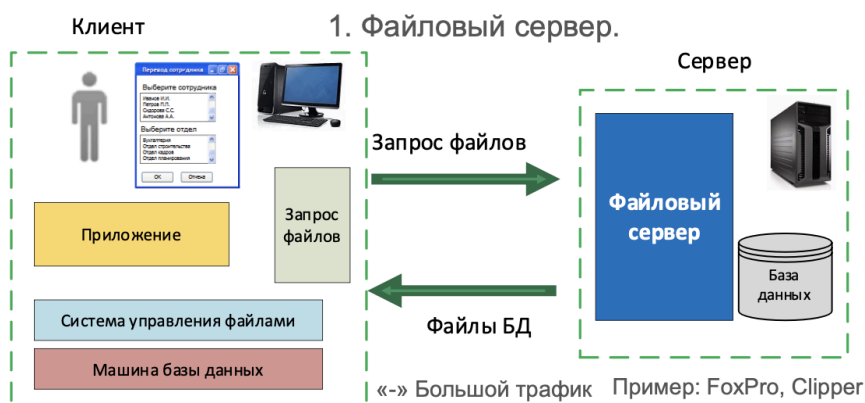


21. Современная сетевая архитектура Клиент сервер.

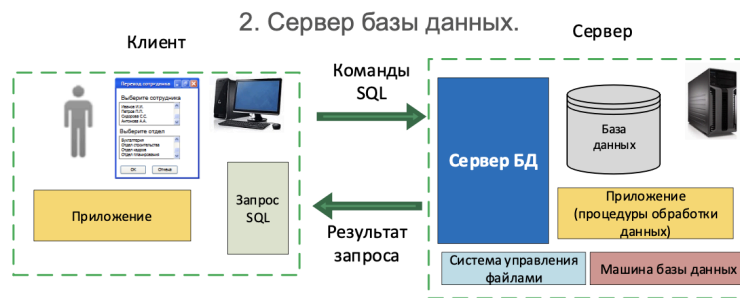
Схема.

Архитектура «клиент-сервер» — это способ организации работы программ, при котором задачи распределены между поставщиком ресурсов (сервером) и потребителем (клиентом).

Существует 3 типа клиент серверного взаимодействия: **Файловый сервер**, в которой сервер выполняет роль пассивного хранилища, предоставляя клиентам доступ к файлам базы данных по сети. Основная вычислительная нагрузка ложится на сторону клиента, где установлены само приложение, система управления файлами и машина базы данных. Когда пользователю нужны данные, клиентская машина запрашивает файлы целиком, что порождает проблему большого сетевого трафика, так как для поиска одной строки серверу приходится пересылать массивные файлы БД. Такая модель была характерна для систем вроде FoxPro и Clipper. (by Google Gemini)



Сервер базы данных, в которой модель переносит тяжелые вычислительные операции по обработке данных с компьютера пользователя на специализированный сервер. Клиент отправляет на сервер не запрос на файлы, а конкретные команды на языке SQL, которые интерпретируются сервером БД. На стороне сервера работают собственная машина базы данных и система управления файлами, которые обрабатывают запрос локально и отправляют клиенту только итоговый результат. Это значительно снижает нагрузку на сеть и позволяет эффективно работать с крупными СУБД, такими как Oracle, MS SQL Server или MySQL. (analyzed by Google Gemini)



Пример: Oracle, MS SQL Server, DB2, MySQL

Сервер приложений, которая добавляет промежуточный слой между пользователем и базой данных, разделяя систему на интерфейс, бизнес-логику и хранилище. Клиент взаимодействует с сервером приложений, отправляя команды вызова методов, а сервер приложений, в свою очередь, обменивается SQL-командами с сервером БД. Это позволяет гибко распределять приложение: пользовательский интерфейс реализуется на стороне клиента, а сложные процедуры обработки и логика программы сосредотачиваются на выделенных серверах. Такая структура обеспечивает высокую безопасность и масштабируемость современных информационных систем. (analyzed by Google Gemini)



22. Понятие сбоя в информационной системе. Типы сбоев.

Мягкие и жесткие сбои.

Сбой в информационной системе — это событие, при котором система или ее компонент перестают корректно выполнять свои функции, что приводит к прерыванию процесса обработки данных.

Типы сбоев и способы защиты от них

1. **Сбой запроса.** СУБД откатывает результаты запроса, формирует код ошибки и возвращает управление программе пользователя.
2. **Сбой приложения.** СУБД откатывает все транзакции и освобождает все ресурсы приложения.
3. **Сбой процесса (копии) сервера.** Процесс сервера завершается, происходит автоматический откат всех незавершенных транзакций в этом процессе.
4. **Сбой корневого процесса сервера.** Сервер прекращает выполнение программ, восстановление требует участия администратора.
5. **Ошибки пользователя.** В зависимости от типа ошибки и от привилегий пользователя устранение требует участия администратора.

Мягкий сбой характеризуется утратой оперативной памяти системы.

Жесткий сбой (аварийный отказ аппаратуры) характеризуется повреждением внешних носителей памяти.

23. Резервное копирование Дамп базы данных. Назначение.

Место хранения. Периодичность выполнения.

Резервное копирование — периодическое сохранение файлов БД на внешнем запоминающем устройстве (облако, флешка, другой жесткий диск, вообще без разницы лишь бы не на вашем ПК и диске, на котором установлена ОС). Выполняется в не противоречивом (согласованном) состоянии БД. **Назначение:** возможность восстановления информации после отказа (сбоя) системы.

Существуют различные типы резервного копирования:

- По объему:
 - полное;
 - частичное инкрементное.
- По частоте:
 - еженедельное;
 - ежедневное.
- Кем/как выполняется:
 - администратором системы вручную;
 - автоматически по расписанию.

Данные, между последним резервным копированием и сбоем, будут утеряны при восстановлении!

Примеры создания дампа

Пояснение: во время создания дампа БД будет явно или неявно остановлена.

mysql:

```
mysqldump [options] db_name [tbl_name ...]  
mysqldump [options] --databases db_name ...  
mysqldump [options] --all-databases
```

`mysqldump -u user_name -p database_name > database_dump_file.sql` (полный дамп)

`mysqldump -u user_name -p --no-data database_name > dumpfile_name.sql` (только структура БД)

postgresql:

`pg_dump -U user_name database_name > backup_file.sql`

Пример восстановления БД из копии

Перед восстановлением из копии необходимо удалить старые данные.

mysql:

```
mysql -u user_name -p database_name < dumpfile_name.sql
```

posgresql:

```
psql -U user_name -d database_name -f dumpfile_name.sql
```

Восстанавливать БД стоит в монопольном режиме с правами администратора!

24. Журнал. Назначение журнала. Метод сохранения данных в журнале. Восстановление после мягкого сбоя.

Журнал транзакций — это резервная копия в СУБД, которая используется для полного восстановления данных.

Журналирование изменений — это функция СУБД, которая сохраняет информацию, необходимую для восстановления БД в предыдущее согласованное состояние в случае логических или физических отказов.

Объем данных для журналирования:

- порядковый номер, тип и время изменения;
- идентификатор транзакции;
- объект, подвергшийся изменению;
- предыдущее и новое состояние объекта.

Восстановить данные можно с помощью *индивидуального отката транзакции*:

1. Выбирается очередное действие из списка транзакции.
2. Выполняется противоположная по смыслу операция: INSERT - DELETE, переворот данных в UPDATE.
3. Все операции журналируются.
4. При успешном откате в журнал заносится запись о конце транзакции.

Восстановление после мягкого сбоя

- нарушилась целостность страниц данных в ОП, данные на диске нарушены,
- сохранился журнал



Физическая согласованность — это момент, когда данные на диске и в ОП одинаковые.

25. Разграничение доступа. Операторы создания пользователей, назначения и отмены прав доступа.

В MySQL управление доступом осуществляется с помощью ключевых операторов SQL.

1) Создание пользователя

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

username - имя пользователя

host - хост, с которого разрешено подключение (например localhost или конкретный ip)

password - пароль пользователя

Пример:

```
CREATE USER 'vovaskavkaza'@'localhost' IDENTIFIED BY 'basketbol228';
```

2) Назначение прав

```
GRANT privilege_type [(column_list)]
```

```
ON database_name.table_name
```

```
TO 'username'@'host';
```

Основные типы привилегий:

ALL PRIVILEGES - полный доступ к указанной БД/таблице

SELECT, INSERT, UPDATE, DELETE - стандартные DML-операции (операции манипулирования данными Data Manipulation Language)

CREATE, DROP, ALTER - управление структурой БД

USAGE - "нулевые" права (подключился и ничего не делаешь)

Примеры:

```
GRANT ALL PRIVILEGES ON mydb.* TO 'user1'@'localhost';
```

```
GRANT SELECT, INSERT ON mydb.users TO 'user2'@'localhost';
```

GRANT CREATE ON mydb.* TO 'user1'@'localhost'; - разрешаем создавать таблицы в конкретной БД

GRANT CREATE ON *.* TO 'admin'@'localhost'; - разрешаем создавать новые базы данных

GRANT ALTER ON mydb.* TO 'user1'@'localhost'; - разрешить изменять структуру таблиц в БД (ALTER TABLE mydb.users ADD COLUMN phone VARCHAR(15);)

GRANT DROP ON *.* TO 'adminuser'@'localhost'; - разрешить удалять базы данных через дроп датабейз

Подробнее про usage:

Используется автоматически при создании пользователя без GRANT

3) Отмена прав

```
REVOKE privilege_type [(column_list)]  
ON database_name.table_name  
FROM 'username'@'host';
```

Примеры:

Отозвать все права на mydb

```
REVOKE ALL PRIVILEGES ON mydb.* FROM 'user'@'localhost';
```

4) Удаление пользователя

```
DROP USER 'username'@'host';
```

5) Просмотр прав пользователя

```
SHOW GRANTS FOR 'username'@'host';
```

Есть ещё команда **FLUSH PRIVILEGES**; - перечитывает таблицы привилегий из системной базы данных mysql и обновляет кэш прав в памяти сервера. Её нужно выполнять только когда редактировал системные таблицы как ебанько. Например:

– ПЛОХОЙ (устаревший) способ создания пользователя:

```
INSERT INTO mysql.user (Host, User, authentication_string, ...)  
VALUES ('localhost', 'newuser', PASSWORD('pass'), ...);
```

– После этого ОБЯЗАТЕЛЬНО нужно:

```
FLUSH PRIVILEGES;
```


26. Функции и процедуры. Назначение. Синтаксис вызова.

в презентации как всегда НАСРАНО 😊🔥

Хранимые подпрограммы - процедуры, функции (ещё триггеры)

Хранимые подпрограммы:

- хранятся на сервере
- являются объектами БД
- включены в функционирование СУБД

Содержат:

- операторы процедурного языка (DECLARE, SET, SELECT...INTO, CASE...WHEN...)
- SQL запросы

Результат: (по сути это и есть назначение)

- процедура - выполнение действий
- функция - возврат значения

Цели:

- повышение производительности
- экономия памяти
- повышение безопасности
- поддержание целостности

Создание процедуры:

CREATE

[DEFINER = user]

PROCEDURE [IF NOT EXISTS] sp_name ([proc_parameter[, ...]])

[characteristic ...] routine_body

[DEFINER = user] - владелец процедуры (по умолчанию текущий пользователь)

[IF NOT EXISTS] - если процедура уже существует - не создавать

sp_name - имя процедуры

[proc_parameter] - входные и выходные параметры

[characteristic] - свойства

routine_body - тело процедуры (внутри BEGIN...END)

Создание функции:

CREATE

[DEFINER = user]

FUNCTION [IF NOT EXISTS] sp_name ([func_parameter[, ...]])

RETURNS type

[characteristic ...] routine_body

про возвращаемые значения:

у функции RETURN (обязательно)

у процедуры - может ничего не возвращать, или возвращать через OUT параметры

Параметры

Для процедуры:

proc_parameter:

[IN | OUT | INOUT] param_name type

IN - входной параметр (по умолчанию, если не указано)

OUT - выходной параметр (возвращаемое из процедуры значение)

INOUT - входной и выходной одновременно

Для функции:

func_parameter:

param_name type

(входные параметры)

СВОЙства:

characteristic:{

COMMENT 'string'

| LANGUAGE SQL

| [NOT] DETERMINISTIC

| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }

| SQL SECURITY { DEFINER | INVOKER }

}

COMMENT 'string' - комментарий к процедуре/функции

LANGUAGE SQL - утверждение, что тело написано на SQL

[NOT] DETERMINISTIC:

DETERMINISTIC - при одинаковых входных параметрах всегда возвращает один и тот же результат

NOT DETERMINISTIC - результат может меняться

Способ обработки процедуры:

CONTAINS SQL - не читает и не изменяет данные

NO SQL - совсем не содержит SQL-операторов

READS SQL DATA - читает данные, но не изменяет (по умолчанию для функций)

MODIFIES SQL DATA - изменяет данные (разрешено только для процедур)

SQL SECURITY определяет от чьего имени выполнится процедура или функция:

DEFINER - от имени создателя процедуры (по умолчанию)

INVOKER - от имени вызывающего пользователя (у него должны быть права на это)

Вызов процедуры: CALL proc_name()

Вызов функции: SELECT func_name()

Последовательность операторов заключается в программные скобки BEGIN...END

Возможности процедур:

- хранимые процедуры могут содержать инструкции DDL
- хранимые процедуры могут содержать SQL-инструкции управления транзакциями

Ограничения функций:

- хранимые функции не должны содержать инструкции DDL
- в хранимых функциях не должно быть запросов, возвращающих наборы результатов
- хранимая функция должна содержать оператор RETURN и возвращать скалярный результат

Примеры наконецто нахуй:

Функция, которая по заданным фамилии и имени возвращает фамилию и инициалы

```
CREATE FUNCTION name_second_name(name varchar(30), second_name varchar(30))
  RETURNS varchar(30)
  BEGIN
    RETURN CONCAT( second_name, ' ', SUBSTRING(name, 1, 1), '. ' );
  END;
```

Вызов

```
SELECT Name, 'Second name', name_second_name(Name, 'Second name')
FROM composer
WHERE id_composer between 10 and 20;
```

Процедура, которая по заданному id_composer и id_musical_composition обновляет данные в таблице plays.

```
CREATE PROCEDURE comp_genre(i INT, j INT)  
BEGIN  
    UPDATE plays INNER JOIN gm g on plays.id_genre = g.id_genre  
    SET quantity = quantity + 1  
    WHERE plays.id_composer = i  
        AND g.id_musical_composition = j  
        AND plays.id_genre = g.id_genre;  
END;
```

Вызов функции из триггера

```
CREATE TRIGGER musical_composition_select  
    AFTER UPDATE  
    on musicdatabase.musical_composition  
    FOR EACH ROW  
BEGIN  
    CALL comp_genre(NEW.id_composer , NEW.id_musical_composition);  
end;
```

27. Операторы присваивания, программных скобок и объявления переменных.

Операторы управления выполнением программы:

Оператор присваивания (установки значений) SET

Присваивает значение переменной или системной переменной

```
SET variable = expr [, variable = expr] ...

variable: {
    user_var_name
  | param_name
  | local_var_name
  | {GLOBAL | @@GLOBAL.} system_var_name
  | {PERSIST | @@PERSIST.} system_var_name
  | {PERSIST_ONLY | @@PERSIST_ONLY.} system_var_name
  | [SESSION | @@SESSION. | @@] system_var_name
}
```

Переменные:

- локальные - объявлены в подпрограмме (DECLARE, хранится только в текущем блоке BEGIN...END)

SET @name = 'полинё';

- глобальные - объявлены в СУБД

SET GLOBAL sort_buffer_size = 100000;

- сессионная переменная (текущее соединение)

SET @@sort_buffer_size = 100000;

Можно так присвоить результат запроса 🤔🤔

SET @total_tax = (SELECT SUM(tax) FROM taxable_transactions);

Оператор программных скобок BEGIN...END

Группирует несколько SQL-операторов в один блок. Обязателен при создании процедур, функций, триггеров

[begin_label:] BEGIN

[statement_list]

END [end_label]

begin_label - ytj,zpfntkmyfz vtnrf блять необязательная метка начала блока

end_label - необязательная метка конца блока (должна совпадать с begin_label)

Пример:

```
DELIMITER //
```

```
CREATE PROCEDURE Hello_World()
```

```
BEGIN
```

```
    SELECT 'Hello, world!';
```

```
END //
```

```
DELIMITER ;
```

нахуй нужен DELIMETER:

По умолчанию MySQL использует ; как символ окончания команды. Но внутри

процедуры мы пишем несколько инструкций, каждая из которых заканчивается ;.

Чтобы сервер не "сломался" на первом ;, нужно временно изменить разделитель.

Объявления переменных DECLARE

Объявляет локально *внутри блока*.

Могут быть объявлены:

- переменные
- условия и драйверы
- курсоры

для тупых(для меня):

тут про локальные переменные — используемые внутри одной подпрограммы,

условия - именованные коды ошибок или предупреждений, которые могут возникнуть при выполнении sql операторов

драйвер — блок кода который автоматически выполняется, когда возникает определенное условие (ошибка, предупреждение, "не найдено")

курсор — объект, который позволяет построчно обрабатывать результат sql запроса (например select)

```
DECLARE var_name [, var_name] ... type [DEFAULT value]
```

Контекст объявления:

- внутри объявленного блока
- внутри вложенных блоков, если не перекрыта

Пример:

```
DELIMITER //
```

```
CREATE PROCEDURE TestDeclare()
```

```
BEGIN
```

```
    DECLARE counter INT DEFAULT 0;
```

```
    DECLARE msg VARCHAR(100) DEFAULT 'Начало';
```

```
    -- Вложенный блок
```

```
    BEGIN
```

```
        DECLARE counter INT DEFAULT 99; -- Перекрывает внешнюю переменную! (это  
типа внутри блока видна только дефолт 99)
```

```
        SET msg = CONCAT('Внутри: ', counter);
```

```
    END;
```

```
    -- Здесь counter снова = 0
```

```
    SELECT msg AS result;
```

```
END //
```

```
DELIMITER ;
```

28. Операторы ветвления.

Условный оператор IF ... ELSEIF ... ELSE ... END IF

Выполняет один из блоков кода в зависимости от истинности условий

```
IF search_condition THEN
    statement_list
[ELSEIF search_condition THEN
    statement_list] ...
[ELSE
    statement_list]
END IF;
```

Пример:

```
DECLARE n INT DEFAULT 5;
DECLARE m INT DEFAULT 3;
DECLARE s VARCHAR(20);
IF n > m THEN
    SET s = '>';
ELSEIF n = m THEN
    SET s = '=';
ELSE
    SET s = '<';
END IF;
SELECT s AS result;
```

– Выведет: '>'

Может присутствовать:

- условные альтернативные (ELSEIF)
- безусловная альтернативная ветка (ELSE)

Это значит что можно писать IF с ELSEIF и/или ELSE, но они необязательны

Оператор варианта CASE ... WHEN ... ELSE ... END CASE

Выбирает действие на основе значения переменной или результата выражения

MySQL поддерживает 2 типа CASE:

- По переменной (значению):

```
CASE case_value
    WHEN when_value THEN statement_list
    [WHEN when_value THEN statement_list] ...
```



```
[ELSE statement_list]
END CASE;
```

Пример:

```
DECLARE grade CHAR(1) DEFAULT 'B';
CASE grade
  WHEN 'A' THEN SELECT 'Отлично!';
  WHEN 'B' THEN SELECT 'Хорошо!';
  WHEN 'C' THEN SELECT 'Удовлетворительно!';
  ELSE SELECT 'Как всегда дрисня брат!';
END CASE;
```

- По условиям:

```
CASE
  WHEN search_condition THEN statement_list
  [WHEN search_condition THEN statement_list] ...
  [ELSE statement_list]
END CASE;
```

Пример:

```
DECLARE score INT DEFAULT 85;
CASE
  WHEN score >= 90 THEN SELECT 'A';
  WHEN score >= 80 THEN SELECT 'B';
  WHEN score >= 70 THEN SELECT 'C';
  ELSE SELECT 'F';
END CASE;
```

Метки(они не были отмечены как операторы ветвления, поповчи просто поделился инфой) i love porov)

Дают имя блоку (BEGIN...END), циклу (LOOP, WHILE, REPEAT) чтобы можно было управлять переходом внутри него. Используются для передачи управления в программных блоках.

```

1  [begin_label:] BEGIN
2  |    statement_list
3  END [end_label]
4
5  [begin_label:] LOOP
6  |    statement_list
7  END LOOP [end_label]
8
9  [begin_label:] WHILE search_condition DO
10 |    statement_list
11 END WHILE [end_label]
12
13 [begin_label:] REPEAT
14 |    statement_list
15 UNTIL search_condition
16 END REPEAT [end_label]

```

Пример использования меток

```

1  DELIMITER //
2  CREATE PROCEDURE TestLabels()
3  BEGIN
4      outer_loop: LOOP
5          DECLARE i INT DEFAULT 0;
6          inner_loop: WHILE i < 5 DO
7              SET i = i + 1;
8              IF i = 3 THEN
9                  ITERATE inner_loop; -- Переходим к следующей итерации внутреннего цикла
10             END IF;
11             SELECT CONCAT('i = ', i);
12             IF i = 4 THEN
13                 LEAVE outer_loop; -- Выходим из внешнего цикла
14             END IF;
15         END WHILE inner_loop;
16     END LOOP outer_loop;
17 END //
18 DELIMITER ;

```

29. Операторы циклов.

Позволяют многократно выполнять блок кода внутри хранимых процедур, функций или триггеров, пока не будет выполнено условие выхода.

Оператор цикла **LOOP...END LOOP** (zaLOOPa 😊)

Бесконечный цикл - выполняется до тех пор, пока не будет явно прерван оператором LEAVE.

```
[begin_label:] LOOP
    statement_list
END LOOP [end_label];
```

ITERATE label - переход к началу цикла с меткой label

LEAVE label - выход из цикла с меткой label

Пример:

```
1 DELIMITER //
2 CREATE PROCEDURE LoopExample()
3 BEGIN
4     DECLARE p1 INT DEFAULT 0;
5
6     label1: LOOP
7         SET p1 = p1 + 1;
8
9         IF p1 < 10 THEN
10             ITERATE label1; — продолжаем цикл
11         END IF;
12
13         LEAVE label1; — выходим из цикла
14     END LOOP label1;
15
16     SELECT p1 AS result; — Выведет: 10
17 END //
18 DELIMITER ;
```

Оператор с предусловием **WHILE ... END WHILE**

Выполняет тело цикла пока условие истинно. Условие проверяется перед каждой итерацией.

```
[begin_label:] WHILE search_condition DO
    statement_list
END WHILE [end_label];
```

Пример:

```
1 DELIMITER //
2 CREATE PROCEDURE WhileExample()
3 BEGIN
4     DECLARE v1 INT DEFAULT 5;
5
6     WHILE v1 > 0 DO
7         SELECT CONCAT('Осталось: ', v1) AS message;
8         SET v1 = v1 - 1;
9     END WHILE;
10
11     SELECT 'Цикл завершён!' AS final_message;
12 END //
13 DELIMITER ;
```

Если условие цикла изначально ложно - тело цикла не выполнится ни разу. Да и пошло оно нахуй.

Оператор с постусловием REPEAT ... UNTIL ... END REPEAT

Выполняет тело цикла минимум один раз, а затем повторяет пока условие не станет истинным. Условие проверяется в конце итерации.

ОБРАТИТЕ ВНИМАНИЕ: !!

Условие проверяется в конце итерации.

да ну нахуй дядь реально???

⚠ Обратите внимание: цикл **продолжается**, пока условие **ЛОЖНО**. Как только оно становится **истинным** — цикл останавливается.

```
[begin_label:] REPEAT
    statement_list
UNTIL search_condition
END REPEAT [end_label];
```

Пример loool:

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE RepeatExample()
```

```
3 BEGIN
```

```
4     DECLARE x INT DEFAULT 0;
```

```
5     DECLARE p1 INT DEFAULT 5;
```

```
6
```

```
7     REPEAT
```

```
8         SET x = x + 1;
```

```
9         SELECT CONCAT('x = ', x) AS iteration;
```

```
10    UNTIL x > p1
```

```
11    END REPEAT;
```

```
12
```

```
13    SELECT 'Цикл закончен' AS result;
```

```
14 END //
```

```
15 DELIMITER ;
```

30. Индексы. Назначение. Синтаксис команд управления индексами.

По простому:

Индекс - вспомогательная структура данных, которая ускоряет поиск и сортировку записей в таблице (это типа как оглавление в книге)

Без индекса СУууууубобратикБД вынуждена выполнять полное сканирование таблицы (FULL SCAN) - перебирать все строки, что медленно при больших объемах данных.

—

Напоминания:

Ключ - атрибут или группа атрибутов, которые позволяют классифицировать (отделить от остальных или объединить в какую-то группу) кортеж отношения.

id	name	question
1	МайклПутята	

Например есть таблица самых позорных вопросов в мкн2. id, name и question это атрибуты (тип столбики). Каждая строка - кортеж. Отношение - совокупность всех кортежей и заголовка(таблица). Вроде так.

—

Для задротов (из лекции):

Индекс - структура, определяющая соответствие значения ключа записи атрибута или группы атрибутов и местоположения этой записи - ключа базы данных.

Назначение

Идея - создать служебную структуру для ускорения выполнения операций.

Нам требуется быстрый доступ к данным при:

- поиске значений (например WHERE dlinabolta = 10 and kolvodopsi > 10)
- сортировке (order by)
- соединении таблиц (наверное индекс тут чтоб быстро искать айди во второй таблице)

Доступ к данным:

- константный $O(1)$
- логарифмический $O(\log n)$
- последовательный $O(n)$

Обеспечивается:

- прямым доступом
- построением дерева
- не требует доп инструкций

Каждому кортежу в БД присваивается внутренний ключ БД - адрес записи в памяти компьютера. Как раз этот ключ и хранит индекс (как я помню по прошлому году: индексируем по столбцу 'прическа' и внутри индекса будет храниться 'мефедроноваячелка':перечисление внутренних ключей, 'лысая бошка дай пирожка моторин':перечисление внутренних ключей, 'взлетная полоса': перечисление внутренних ключей). Внутренние ключи - адреса в памяти.

fuck

Организация индекса:

- хранится в отдельном файле или отдельной области памяти (не смешивается с данными)
- пустые значения атрибутов не индексируются (на NULL поебать)
- не влияет на размещение данных в таблице(индекс только указывает на данные)
- у меня болит поясница(((
- поддерживаются динамически
- один индекс -> одна таблица
- для одной таблицы можно сделать несколько индексов

Реализация индекса:

- содержит упорядоченные последовательности индексируемого атрибута - быстрый поиск
- занимает меньше памяти - быстрый поиск (вроде хранит только ключи и указатели)

Индексы бывают:

- первичными - создаются по уникальным данным (например id)
- вторичные - не по уникальным, могут содержать дубликаты
- простые - по одному атрибуту
- составные - по набору атрибутов
- линейные - списки (отсортированный список ключей)
- многоуровневые деревья (Hash, B+)

◆ Синтаксис создания индекса

sql



```
1 CREATE [UNIQUE | FULLTEXT | SPATIAL] INDEX index_name
2 [USING {BTREE | HASH}]
3 ON tbl_name (key_part,...)
4 [index_option]
```

Где:

- **key_part**: col_name [(length)] или выражение.
- **index_option**: COMMENT, VISIBLE, INVISIBLE, KEY_BLOCK_SIZE, и т.д.
- **algorithm_option**: ALGORITHM = {DEFAULT | INPLACE | COPY}
- **lock_option**: LOCK = {DEFAULT | NONE | SHARED | EXCLUSIVE}

Создание простого индекса:

```
CREATE INDEX city_district ON city(District);
```

Удаление индекса:

```
DROP INDEX city_district ON city;
```

Просмотр индексов:

```
SHOW INDEX FROM city;
```

– или

```
SHOW KEYS FROM city;
```


Использование индекса:

Для сортировки:

```
SELECT Name FROM city ORDER BY CountryCode;
```

– Если есть индекс по CountryCode — будет быстрая сортировка

Для соединения:

```
SELECT city.Name, country.Name
```

```
FROM city JOIN country ON city.CountryCode = country.Code;
```

– Если есть индекс по CountryCode — быстрое соединение

Для полнотекстового поиска:

Конструкция MATCH-AGAINST

```
CREATE FULLTEXT INDEX ft_title_body ON articles(title, body);
```

```
SELECT *, MATCH(title, body) AGAINST('database') AS REL
```

```
FROM articles
```

```
WHERE MATCH(title, body) AGAINST('database') > 0;
```

i love popov so much



i love popov<3<3<3

все я всё прочитала sosať

ура я тоже